

Clase NoSql

Guía paso a paso: trabajar con colecciones y consultas en MongoDB Compass

1. Propósito de esta guía

Esta guía está diseñada para que el estudiante pueda seguir un proceso ordenado después de haber visto los videos introductorios sobre JSON, NoSQL, instalación de MongoDB Compass, creación de bases de datos y creación de colecciones.

Aquí aprenderá, de manera práctica y detallada:

- cómo agregar documentos a una colección,
- cómo crear varias colecciones dentro de una base de datos,
- cómo relacionar la estructura de los datos con el funcionamiento del negocio,
- cómo realizar consultas básicas,
- cómo filtrar información,
- cómo ordenar resultados,
- cómo modificar y eliminar datos.

La idea es que el estudiante no solo “haga clic”, sino que comprenda **qué está haciendo y por qué lo está haciendo**.

2. Conceptos básicos antes de empezar

En MongoDB, la información se organiza así:

- **Base de datos:** contiene un conjunto de colecciones.
- **Colección:** contiene varios documentos.
- **Documento:** es la unidad principal de información. Se parece a un objeto JSON.

Un documento puede representar, por ejemplo:

- un cliente,
- un producto,
- una venta,
- un estudiante,
- un pedido,
- un profesor,
- una cita médica.

MongoDB trabaja muy bien cuando los datos no siempre tienen exactamente la misma forma, porque permite guardar información flexible.

3. Ejemplo de negocio para la guía

Para que el trabajo tenga sentido, usaremos un ejemplo sencillo de una **tienda**.

En una tienda pueden existir varias colecciones, por ejemplo:

- **clientes:** información de las personas que compran.
- **productos:** información de los artículos que se venden.
- **ventas:** registros de compras realizadas.
- **categorías:** clasificación de los productos.

Esto es importante porque en un negocio real no se guarda todo en una sola colección. Cada colección representa un grupo de datos con un propósito específico.

4. Cómo agregar datos a una colección

4.1 Entrar a MongoDB Compass

1. Abra MongoDB Compass.
2. Conéctese a su servidor de MongoDB.
3. Ingrese a la base de datos que creó previamente.

4. Abra la colección donde desea guardar información.

Si la colección todavía no tiene documentos, verá la opción para insertar el primero.

4.2 Insertar un documento manualmente

Para agregar datos a una colección:

1. Entre a la colección deseada.
2. Haga clic en **Insert Document**.
3. Se abrirá un editor en formato JSON.
4. Escriba el documento con la estructura correcta.
5. Presione **Insert** o **Save**.

Ejemplo de documento para la colección **clientes**

```
{
  "nombre": "Juan Pérez",
  "documento": "1002456789",
  "correo": "juan.perez@email.com",
  "telefono": "3001234567",
  "ciudad": "Pasto",
  "estado": "activo"
}
```

Este documento representa un cliente. Cada campo tiene un significado:

- **nombre**: nombre completo del cliente.
- **documento**: número de identificación.
- **correo**: email de contacto.
- **telefono**: número telefónico.
- **ciudad**: lugar de residencia.
- **estado**: indica si el cliente está activo o inactivo.

4.3 Insertar varios documentos

Una colección no debe tener solo un documento. Debe tener varios registros para que las consultas tengan sentido.

Ejemplo de varios clientes:

```
[
  {
    "nombre": "Juan Pérez",
    "documento": "1002456789",
    "correo": "juan.perez@email.com",
    "telefono": "3001234567",
    "ciudad": "Pasto",
    "estado": "activo"
  },
  {
    "nombre": "María Gómez",
    "documento": "1023456781",
    "correo": "maria.gomez@email.com",
    "telefono": "3109876543",
    "ciudad": "Ipiiales",
    "estado": "activo"
  },
  {
    "nombre": "Carlos Ruiz",
    "documento": "1098765432",
    "correo": "carlos.ruiz@email.com",
    "telefono": "3155551122",
    "ciudad": "Pasto",
    "estado": "inactivo"
  }
]
```

4.4 Reglas para escribir documentos correctamente

Al momento de agregar datos, el estudiante debe tener en cuenta:

- Los nombres de los campos deben ser claros y coherentes.
- Se recomienda usar siempre el mismo estilo para los nombres, por ejemplo `nombre`, `fecha_nacimiento`, `precio`.
- Los valores de texto van entre comillas.
- Los números no llevan comillas si realmente son numéricos.
- Los documentos pueden tener campos diferentes, pero es mejor mantener una estructura parecida.
- El contenido debe reflejar la realidad del negocio.
- MongoDB crea automáticamente el campo `_id` para cada documento. En la mayoría de los casos, el estudiante **no debe escribirlo manualmente**.

Sobre el campo `_id`

Cada documento en MongoDB tiene un identificador único llamado `_id`.

Este campo sirve para distinguir un documento de otro, aunque los demás datos sean iguales.

Ejemplo:

```
{
  "_id": ObjectId("65f1c8a1e4b0a1d2c3f4a567"),
  "nombre": "Juan Pérez",
  "documento": "1002456789",
  "correo": "juan.perez@email.com"
}
```

Cuando el documento se inserta desde MongoDB Compass, el sistema puede generar ese `_id` de forma automática.

¿Debe el estudiante escribir `_id` ?

Depende del ejercicio:

- **Si el profesor no pide nada especial**, lo más recomendable es **no escribir** `_id` y dejar que MongoDB lo cree automáticamente.

- **Si el profesor quiere que el estudiante pratique**, entonces sí puede escribirlo manualmente, pero debe ser un valor único.

Ejemplo de `_id` escrito manualmente

```
{
  "_id": 1,
  "nombre": "Juan Pérez",
  "documento": "1002456789",
  "correo": "juan.perez@email.com"
}
```

También puede usarse un valor más completo, pero siempre debe evitarse repetirlo en otro documento.

Recomendación para los estudiantes

En esta guía se recomienda que, al llenar colecciones por primera vez, el estudiante se concentre en los campos del negocio, como `nombre`, `precio`, `correo` o `fecha`, y deje que MongoDB genere el `_id` automáticamente.

5. Cómo tener varias colecciones en una misma base de datos

Una base de datos puede contener varias colecciones porque un negocio maneja varios tipos de información.

5.1 Ejemplo de estructura para la base de datos `tienda`

- `clientes`
- `productos`
- `ventas`
- `categorias`

Cada colección cumple una función distinta.

5.2 ¿Por qué no guardar todo en una sola colección?

No es recomendable mezclar toda la información en una sola colección porque:

- dificulta la organización,
- complica las consultas,
- hace más difícil actualizar la información,
- genera datos desordenados,
- vuelve el sistema menos entendible.

5.3 Ejemplo de qué guarda cada colección

Colección **clientes**

Guarda información de las personas que compran.

```
{
  "nombre": "Juan Pérez",
  "documento": "1002456789",
  "correo": "juan.perez@email.com"
}
```

Colección **productos**

Guarda información de los artículos disponibles.

```
{
  "nombre": "Teclado mecánico",
  "precio": 180000,
  "stock": 25,
  "categoria": "periféricos"
}
```

Colección **ventas**

Guarda los registros de compra.

```
{
  "cliente": "Juan Pérez",
  "fecha": "2026-03-26",
  "total": 180000
}
```

Colección **categorias**

Guarda la clasificación de productos.

```
{
  "nombre": "Periféricos",
  "descripcion": "Dispositivos de entrada y salida para el co
mputador"
}
```

6. Cómo decidir qué colecciones necesita un negocio

Para crear una base de datos bien pensada, el estudiante debe leer el negocio y responder:

- ¿Qué personas intervienen?
- ¿Qué productos, servicios o recursos maneja?
- ¿Qué transacciones realiza?
- ¿Qué información necesita consultar el usuario?
- ¿Qué datos deben guardarse por separado?

Ejemplo de análisis

Si el negocio es una tienda, normalmente necesita guardar:

- clientes,
- productos,
- ventas,

- categorías,
- proveedores.

Si el negocio es una academia, podría necesitar:

- estudiantes,
- profesores,
- cursos,
- matrículas,
- notas.

La idea es que el estudiante aprenda a pensar en términos del negocio, no solo de la herramienta.

7. Cómo hacer consultas en MongoDB Compass

MongoDB Compass permite consultar documentos dentro de una colección usando filtros.

Las consultas sirven para responder preguntas como:

- ¿Cuáles clientes están activos?
 - ¿Qué productos cuestan más de cierto valor?
 - ¿Qué ventas hizo un cliente específico?
 - ¿Qué documentos coinciden con una ciudad determinada?
-

7.1 Buscar todos los documentos

Al entrar a una colección, MongoDB muestra todos los documentos.

Esto equivale a consultar toda la colección.

7.2 Filtrar por un valor exacto

Si desea mostrar solo los clientes de una ciudad específica:

```
{ "ciudad": "Pasto" }
```

Esto mostrará únicamente los documentos donde el campo `ciudad` tenga ese valor.

7.3 Filtrar por varios criterios

Si desea buscar clientes que vivan en Pasto y estén activos:

```
{ "ciudad": "Pasto", "estado": "activo" }
```

Esto significa que el documento debe cumplir ambas condiciones al mismo tiempo.

7.4 Buscar por un valor numérico

Si desea encontrar productos con precio de 180000:

```
{ "precio": 180000 }
```

Si el campo es numérico, se debe consultar como número.

7.5 Buscar por comparación

MongoDB permite usar operadores para comparar valores.

Mayor que

```
{ "precio": { "$gt": 100000 } }
```

Muestra productos con precio mayor que 100000.

Menor que

```
{ "precio": { "$lt": 50000 } }
```

Muestra productos con precio menor que 50000.

Mayor o igual que

```
{ "precio": { "$gte": 100000 } }
```

Menor o igual que

```
{ "precio": { "$lte": 50000 } }
```

7.6 Consultar documentos que no tengan un valor específico

Si desea encontrar clientes que no estén activos:

```
{ "estado": { "$ne": "activo" } }
```

`$ne` significa "distinto de".

7.7 Buscar coincidencias parciales en texto

Si desea buscar nombres que contengan una palabra específica, puede usar expresiones regulares.

Ejemplo: clientes cuyo nombre contenga "Juan"

```
{ "nombre": { "$regex": "Juan" } }
```

Esto es útil cuando no se conoce el texto exacto.

8. Cómo ordenar resultados

Ordenar permite ver los datos de forma organizada.

Ejemplo

- ordenar productos por precio de menor a mayor,
- ordenar clientes por nombre alfabéticamente,
- ordenar ventas por fecha.

En MongoDB Compass, esto se hace desde la sección de ordenamiento.

Ejemplo conceptual

- Ascendente: de menor a mayor o de A a Z.
 - Descendente: de mayor a menor o de Z a A.
-

9. Cómo limitar resultados

A veces no se necesitan todos los documentos, solo una parte.

Por ejemplo, puede mostrar solo los primeros 5 productos o los primeros 10 clientes.

Esto ayuda a:

- revisar datos de forma más rápida,
 - evitar saturar la pantalla,
 - comprobar que la consulta funciona.
-

10. Cómo modificar un documento

En ocasiones, el estudiante necesitará corregir un dato.

Por ejemplo:

- cambiar un correo,
- actualizar un estado,
- modificar el precio de un producto,
- corregir un nombre mal escrito.

Pasos generales

1. Buscar el documento.
2. Abrirlo.
3. Cambiar el valor necesario.
4. Guardar los cambios.

Ejemplo

Antes:

```
{  
  "nombre": "Juan Pérez",  
  "estado": "inactivo"  
}
```

Después:

```
{  
  "nombre": "Juan Pérez",  
  "estado": "activo"  
}
```

11. Cómo eliminar un documento

Eliminar un documento significa quitar un registro de la colección.

Esto solo debe hacerse cuando realmente ya no se necesita la información o cuando el dato está incorrecto y debe ser reemplazado.

Pasos generales

1. Buscar el documento.
2. Revisar bien que sea el correcto.
3. Eliminarlo desde la opción correspondiente.
4. Confirmar la acción.

Se recomienda tener cuidado porque esta acción puede no revertirse fácilmente.

12. Ejercicio guía completo

Caso: tienda de tecnología

La tienda necesita guardar información de:

- clientes,
- productos,
- ventas.

Paso 1: crear la base de datos

Nombre sugerido:

```
tienda_tecnologia
```

Paso 2: crear las colecciones

- `clientes`
- `productos`
- `ventas`

Paso 3: insertar documentos en `clientes`

```
{
  "nombre": "Laura Martínez",
  "documento": "1001001001",
  "correo": "laura@email.com",
  "telefono": "3001112222",
  "ciudad": "Cali",
  "estado": "activo"
}
```

```
{
  "nombre": "Pedro López",
  "documento": "1002002002",
  "correo": "pedro@email.com",
  "telefono": "3003334444",
  "ciudad": "Pasto",
}
```

```
"estado": "activo"
}
```

Paso 4: insertar documentos en **productos**

```
{
  "nombre": "Mouse inalámbrico",
  "precio": 45000,
  "stock": 30,
  "categoria": "periféricos"
}
```

```
{
  "nombre": "Monitor 24 pulgadas",
  "precio": 650000,
  "stock": 10,
  "categoria": "pantallas"
}
```

Paso 5: insertar documentos en **ventas**

```
{
  "cliente": "Laura Martínez",
  "producto": "Mouse inalámbrico",
  "cantidad": 2,
  "total": 90000,
  "fecha": "2026-03-26"
}
```

```
{
  "cliente": "Pedro López",
  "producto": "Monitor 24 pulgadas",
  "cantidad": 1,
}
```

```
"total": 650000,  
"fecha": "2026-03-26"  
}
```

13. Consultas de práctica

Consulta 1: mostrar clientes de Pasto

```
{ "ciudad": "Pasto" }
```

Consulta 2: mostrar productos con precio mayor a 50000

```
{ "precio": { "$gt": 50000 } }
```

Consulta 3: mostrar clientes activos

```
{ "estado": "activo" }
```

Consulta 4: mostrar ventas de un cliente específico

```
{ "cliente": "Laura Martínez" }
```

Consulta 5: mostrar productos de la categoría periféricos

```
{ "categoria": "periféricos" }
```

14. Recomendaciones para trabajar mejor

- Use nombres claros para bases de datos, colecciones y campos.
- No mezcle datos que cumplen funciones diferentes.
- Inserte varios documentos para que las consultas sean útiles.

- Revise que los tipos de datos sean coherentes.
 - Piense siempre en el negocio antes de pensar en la herramienta.
 - Practique consultas simples antes de intentar consultas más complejas.
-

15. Actividad

Enunciado

Cree una base de datos para un negocio de su elección y construya al menos tres colecciones. Luego agregue mínimo tres documentos en cada colección y realice al menos cinco consultas diferentes.

Debe incluir

- nombre de la base de datos,
- nombre de las colecciones,
- documentos insertados,
- consultas realizadas,
- explicación breve de qué muestra cada consulta.

Ejemplo de negocios posibles

- peluquería,
 - tienda de ropa,
 - consultorio médico,
 - restaurante,
 - biblioteca,
 - academia de cursos,
 - veterinaria.
-

16. Cierre

MongoDB no solo consiste en crear una base de datos y una colección. El verdadero aprendizaje está en saber:

- qué datos necesita el negocio,
- cómo organizarlos en colecciones,
- cómo guardar documentos correctamente,
- cómo consultar la información,
- cómo mantener los datos ordenados y útiles.

Si el estudiante entiende esto, podrá trabajar con MongoDB de forma mucho más segura y profesional.